École Nationale de la Statistique et de l'Analyse Économique Pierre NDIAYE

Projet Statistique sous R

Aboubacar HEMA
Analyste de recherche
International Food Policy Research Institute (IFPRI)

Ce cours s'inscrit dans la formation des Ingénieurs Statisticiens Économistes (ISE) de l'ENSAE et vise à développer des compétences opérationnelles en conduite de projets statistiques complets à l'aide du logiciel R. En s'appuyant sur des données réelles issues des contextes africains, notamment celles produites par des organisations comme l'IFPRI, la FAO, la Banque Mondiale ou les instituts nationaux de statistique, les étudiant.e.s apprendront à transformer des données brutes en résultats analytiques communicables.
L'approche pédagogique est résolument appliquée : chaque concept théorique est immédiatement mis en pratique dans un projet fil rouge qui progresse au fil des séances, conduisant à la production d'un rapport d'analyse statistique de qualité professionnelle.

Objectifs généraux

- Maîtriser l'environnement R et RStudio pour la conduite de projets statistiques de bout en bout
- Acquérir des réflexes rigoureux en matière de gestion, nettoyage et documentation des données
- Produire des analyses statistiques descriptives et inférentielles solides et reproductibles
- Appliquer des méthodes statistiques avancées adaptées aux données socio-économiques (enquêtes ménages, données agricoles, indicateurs de développement)
- Développer des capacités de visualisation et de communication des résultats à destination de publics variés
- Initier aux bonnes pratiques de la recherche reproductible (R Markdown, Git)

Séance	Thème	Contenu principal
Séance 1	**Environnement R et RStudio**	Installation, packages, projets RStudio, bonnes pratiques, introduction à tidyverse
Séance 2	**Importation & exploration des données**	read_csv, haven, readxl, API, web scraping basique, structure des données, str(), summary()
Séance 3	**Nettoyage et préparation**	dplyr, tidyr, gestion des valeurs manquantes, recodage, jointures, reshape long/wide
Séance 4	**Statistiques descriptives avancées**	Tableaux croisés, gtsummary, pondération d'enquêtes (srvyr/survey), indices composites
Séance 5	**Visualisation avec ggplot2**	Grammaire des graphiques, palettes, thèmes, graphiques publiables, cartes choroplèthes (sf, tmap)
Séance 6	**Inférence statistique & tests**	Tests paramétriques et non-paramétriques, bootstrap, intervalles de confiance, correction multiple
Séance 7	**Reproductibilité & communication**	R Markdown, Quarto, rapports automatisés, tableaux publication-ready (gt, flextable), Git/GitHub
Séance 8	**Présentations des projets**	Soutenance des projets de groupe, retours et discussions

Environnement R & RStudio

Fondations pour un projet statistique robuste

1. Pourquoi R pour le statisticien économiste ?

R s'est imposé comme l'un des langages de référence en statistique appliquée, économétrie et data science. Pour un Ingénieur Statisticien Économiste, sa maîtrise constitue un avantage compétitif majeur sur le marché du travail — aussi bien dans les organisations internationales (Banque Mondiale, PNUD, FAO, IFPRI) que dans les administrations statistiques nationales, les cabinets de conseil ou la recherche académique.

Contrairement à des logiciels comme Stata ou SPSS, R est entièrement libre, gratuit, et dispose d'un écosystème de packages en constante expansion. Chaque année, des milliers de chercheurs et de praticiens publient de nouveaux outils directement utilisables depuis R. La communauté R est également particulièrement active en Afrique, avec des groupes d'utilisateurs dans plusieurs pays de l'UEMOA.

Atout	Pourquoi c'est important pour vous
Open source	Gratuit, transparent, auditable — idéal pour la recherche reproductible
Écosystème riche	Plus de 20 000 packages sur CRAN couvrant tous les domaines de l'analyse statistique
Visualisation	ggplot2 produit des graphiques de qualité publication inégalés
Reproductibilité	R Markdown et Quarto permettent de combiner code, résultats et texte
Communauté	Stack Overflow, RStudio Community, groupes R-Africa — support illimité
Intégration	Interface avec Python, SQL, Spark, Git, LaTeX — outil universel

2. Installation et configuration de l'environnement

2.1 R et RStudio

L'installation se fait en deux temps:d'abord R lui-même (le moteur de calcul), puis RStudio (l'interface de développement intégrée, IDE). RStudio propose une interface à quatre panneaux:l'éditeur de scripts, la console R, l'explorateur d'environnement/historique, et le panneau fichiers/graphiques/aide.

- R:télécharger la dernière version stable depuis cran.r-project.org (actuellement R 4.4.x)
- RStudio Desktop:disponible gratuitement sur posit.co/download/rstudio-desktop
- Configurer l'encodage UTF-8 (Tools > Global Options > Code > Saving)

- Désactiver la sauvegarde automatique du workspace (.RData) pour des sessions propres

2.2 Les packages essentiels

Un package R est une collection de fonctions, données et documentation. L'installation se fait avec install.packages() et le chargement avec library(). La collection tidyverse regroupe les packages les plus utilisés pour l'analyse de données moderne.

```
# Installation du tidyverse et des packages du cours
install.packages("tidyverse")      # Collection principale:ggplot2, dplyr, tidyr, readr...
install.packages("haven")          # Import données Stata (.dta) et SPSS (.sav)
install.packages("readxl")         # Import fichiers Excel
install.packages("janitor")        # Nettoyage des noms de colonnes
install.packages("gtsummary")      # Tableaux statistiques publication-ready
install.packages("survey")         # Analyses pondérées (sondages)
install.packages("sf")             # Données spatiales (shapefiles)
install.packages("renv")           # Gestion des dépendances du projet
```

3. Organisation d'un projet R

L'un des premiers réflexes à acquérir est d'organiser son travail sous forme de projets RStudio (.Rproj). Un projet définit un répertoire de travail, isole les environnements et facilite la collaboration. Voici la structure recommandée pour ce cours :

```
mon_projet/
├── mon_projet.Rproj       # Fichier projet RStudio
├── data/
│   ├── raw/               # Données brutes (JAMAIS modifiées)
│   └── processed/         # Données nettoyées et transformées
├── R/
│   ├── 01_import.R        # Scripts numérotés par ordre d'exécution
│   ├── 02_nettoyage.R
│   ├── 03_analyse.R
│   └── fonctions.R        # Fonctions personnalisées réutilisables
├── output/
│   ├── figures/           # Graphiques exportés
│   └── tables/            # Tableaux exportés
├── rapport/
│   └── rapport_final.qmd  # Document Quarto (rapport automatisé)
└── README.md              # Description du projet
```

4. Introduction au tidyverse et à la philosophie tidy

Le tidyverse est une collection cohérente de packages R conçue autour d'une philosophie unifiée de manipulation et visualisation des données. La notion centrale est celle de tidy data (données ordonnées):chaque variable occupe une colonne, chaque observation une

ligne, chaque valeur une cellule. Cette structure facilite toutes les opérations analytiques ultérieures.

4.1 L'opérateur pipe

L'opérateur pipe (|> en R natif, ou %>% avec magrittr) est fondamental dans la programmation tidyverse. Il permet d'enchaîner des opérations de façon lisible, en passant le résultat de l'opération précédente comme premier argument de la suivante. Le code devient ainsi lisible de gauche à droite, comme une phrase.

```
# Sans pipe:difficile à lire
summary(filter(select(read_csv("data/raw/menages.csv"), revenu, region), revenu >
0))

# Avec pipe:fluide et lisible
read_csv("data/raw/menages.csv") |>
  select(revenu, region) |>
  filter(revenu > 0) |>
  summary()
```

5. Gestion des dépendances avec renv

La reproductibilité d'un projet R nécessite de figer les versions des packages utilisés. renv crée un environnement virtuel propre à chaque projet, avec un fichier renv.lock qui enregistre les versions exactes. Cela garantit que votre analyse produira les mêmes résultats dans 6 mois ou sur un autre ordinateur.

```
# Initialiser renv dans un projet
renv::init()

# Après avoir installé de nouveaux packages
renv::snapshot()   # Enregistre l'état actuel dans renv.lock

# Pour restaurer l'environnement sur un autre poste
renv::restore()    # Installe exactement les mêmes versions
```

A faire 1
1. Installer R 4.4.x et RStudio et configurer les options recommandées
2. Créer le projet RStudio du fil rouge avec la structure de dossiers standard
3. Installer tidyverse, haven, readxl, janitor, gtsummary, renv
4. Initialiser renv et réaliser un premier snapshot
5. Charger le jeu de données EHCVM et afficher les premières observations
6. Rédiger un README.md décrivant le projet, les données et les objectifs

Importation & Exploration des données

De la source brute à la première compréhension

1. Les sources de données en statistique du développement

En Afrique, les statisticiens économistes travaillent avec une grande variété de sources de données. Chaque source a ses spécificités techniques et ses contraintes d'accès qu'il est essentiel de maîtriser. Ce module présente les principales sources et leurs modalités d'importation dans R.

Source	Caractéristiques techniques
Enquêtes ménages (EHCVM, LSMS, EDS)	Format Stata ou SPSS, souvent avec des labels de valeurs et des poids de sondage
Données administratives (ANSD, INS)	Souvent en Excel ou CSV, nécessitent un nettoyage important
Bases institutionnelles (FAO, BM, PNUD)	API REST ou fichiers CSV/Excel téléchargeables, packages R dédiés
Données satellitaires & géospatiales	Rasters (MODIS, ERA5) et shapefiles (limites administratives)
Web scraping	Données non structurées depuis des sites web, avec rvest
Bases de données relationnelles	SQL via DBI, RSQLite, RPostgres pour les grands volumes

2. Importation de fichiers locaux

2.1 Fichiers CSV et texte (readr)

readr fait partie du tidyverse et propose des fonctions rapides et robustes pour l'importation de fichiers plats. Il détecte automatiquement les types de colonnes et gère les encodages problématiques.

```
library(tidyverse)

# Import CSV standard
menages <- read_csv("data/raw/ehcvm_menages.csv")

# Avec options avancées
menages <- read_csv(
  "data/raw/ehcvm_menages.csv",
  col_types = cols(
    hhid     = col_character(),    # Identifiant ménage:toujours en caractère
    milieu   = col_factor(),       # Variable catégorielle
    revenu   = col_double(),       # Revenu en FCFA
```

```
    date_enq = col_date(format = "%d/%m/%Y")
  ),
  locale = locale(encoding = "UTF-8", decimal_mark = ",")
)

glimpse(menages)  # Aperçu structuré du data frame
```

2.2 Fichiers Stata et SPSS (haven)

La quasi-totalité des enquêtes ménages africaines sont distribuées en format Stata (.dta). Le package haven importe ces fichiers en préservant les labels de valeurs, les labels de variables, les valeurs manquantes spéciales (tagged NA) et les attributs de format Stata.

```
library(haven)

# Import d'une enquête Stata
menages <- read_dta("data/raw/ehcvm2021_menages.dta")

# Afficher les labels de variables
var_label(menages$milieu)       # "Milieu de résidence"
val_labels(menages$milieu)      # 1 = "Urbain", 2 = "Rural"

# Convertir les haven_labelled en facteurs R
menages <- menages |>
  mutate(across(where(is.labelled), as_factor))

# Valeurs manquantes spéciales Stata (.a, .b, etc.)
menages |> filter(is.na(revenu)) |> nrow()  # NA standard
menages |> filter(is_tagged_na(revenu)) |> nrow()  # NA taggés
```

2.3 Fichiers Excel (readxl)

Les données des instituts nationaux et des ministères sont souvent distribuées en Excel. readxl permet l'import de fichiers .xlsx et .xls avec un contrôle précis des feuilles, plages de cellules et types.

```
library(readxl)

# Lister les feuilles d'un fichier
excel_sheets("data/raw/statistiques_agricoles.xlsx")

# Import d'une feuille spécifique
cereales <- read_excel(
  "data/raw/statistiques_agricoles.xlsx",
  sheet = "Productions_2021",
  range = "A3:H150",      # Plage de cellules (ignorer les en-têtes fusionnées)
  na = c("", "N/D", "-")  # Codes de valeurs manquantes
```

```
)
```

3. Accès aux API institutionnelles

3.1 API de la Banque Mondiale (wbstats)

Le package wbstats donne accès à plus de 16 000 indicateurs de la Banque Mondiale directement depuis R. C'est une source précieuse pour les analyses comparatives inter-pays sur des thématiques comme la pauvreté, l'agriculture, la santé ou l'éducation.

```
library(wbstats)

# Rechercher des indicateurs
wbstats::wb_search("food insecurity") |> select(indicator_id, indicator)

# Télécharger plusieurs indicateurs pour l'Afrique subsaharienne
donnees_bm <- wb_data(
  country   = c("SN", "ML", "BF", "NE", "CI", "GH", "NG"),  # Codes ISO
  indicator = c(
    "SN.ITK.DEFC.ZS",   # Prévalence de la sous-alimentation (%)
    "AG.YLD.CREL.KG",   # Rendement céréalier (kg/ha)
    "SP.POP.TOTL"       # Population totale
  ),
  start_date = 2010,
  end_date   = 2023
)

glimpse(donnees_bm)
```

3.2 FAOSTAT

FAOSTAT est la base de données statistiques de la FAO, couvrant la production agricole, les échanges commerciaux, les prix, la sécurité alimentaire et l'utilisation des terres pour tous les pays du monde. Le package FAOSTAT permet un accès direct depuis R.

```
library(FAOSTAT)

# Télécharger les données de production agricole
production <- get_faostat_bulk(
  code = "QCL",              # Crops and Livestock Products
  data_folder = "data/raw/"
)

# Filtrer pour l'Afrique de l'Ouest
prod_ao <- production |>
  filter(Area %in% c("Senegal", "Mali", "Burkina Faso",
                     "Niger", "Côte d'Ivoire"),
```

```
        Item %in% c("Maize", "Millet", "Sorghum", "Rice"),
        Year >= 2010)
```

4. Exploration initiale d'un jeu de données

Avant toute analyse, une phase d'exploration systématique est indispensable. Elle permet de comprendre la structure des données, d'identifier les problèmes potentiels (valeurs manquantes, aberrantes, incohérences) et d'orienter les étapes de nettoyage. En anglais, cette phase est parfois appelée EDA (Exploratory Data Analysis).

```
# Exploration de base
dim(menages)          # Dimensions:lignes x colonnes
names(menages)        # Noms de toutes les variables
glimpse(menages)      # Types et premières valeurs
summary(menages)      # Statistiques descriptives par variable

# Explorer les valeurs manquantes
library(naniar)
vis_miss(menages)             # Visualisation cartographique des NA
miss_var_summary(menages)     # % de NA par variable
gg_miss_var(menages)          # Graphique des valeurs manquantes

# Distribution des variables catégorielles
menages |> count(milieu, sort = TRUE)
menages |> count(region, milieu)     # Tableau croisé simple

# Distribution des variables numériques
menages |>
  select(where(is.numeric)) |>
  pivot_longer(everything()) |>
  ggplot(aes(x = value)) +
  geom_histogram(fill = "#2E75B6", bins = 30) +
  facet_wrap(~name, scales = "free") +
  theme_minimal()
```

A faire 2
1. Importer le fichier EHCVM ménages (.dta) avec haven, vérifier les labels
2. Importer les données FAOSTAT de production céréalière pour l'Afrique de l'Ouest
3. Télécharger 3 indicateurs Banque Mondiale pour les pays cibles du projet
4. Réaliser une EDA complète:dimensions, types, NA, distributions
5. Produire un rapport HTML automatique avec DataExplorer::create_report()

6. Rédiger dans le rapport de projet une section 'Description des sources de données'

Nettoyage & Préparation des données
Transformer les données brutes en données analytiques

1. La grammaire dplyr pour la manipulation de données

dplyr est le package central du tidyverse pour la manipulation de tableaux de données. Il propose six verbes fondamentaux qui couvrent l'essentiel des opérations de transformation. Ces verbes sont conçus pour être composables via l'opérateur pipe, permettant d'exprimer des transformations complexes de façon claire et lisible.

Verbe dplyr	Rôle
filter()	Sélectionner des lignes selon une ou plusieurs conditions logiques
select()	Sélectionner ou réorganiser des colonnes par nom, position ou type
mutate()	Créer ou modifier des colonnes (transformations, recodages)
summarise()	Agréger des lignes en statistiques résumées (moyenne, total, etc.)
group_by()	Définir des groupes pour que summarise() opère par groupe
arrange()	Trier les lignes par une ou plusieurs variables (croissant ou décroissant)

```
library(tidyverse)

# Pipeline de manipulation complet
analyse_revenu <- menages |>
  filter(milieu != "Autre", revenu > 0, !is.na(revenu)) |>
  select(hhid, region, milieu, revenu, taille_menage, poids) |>
  mutate(
    revenu_pc    = revenu / taille_menage,          # Revenu par tête
    log_revenu   = log(revenu_pc),                  # Transformation log
    quintile     = ntile(revenu_pc, 5),             # Quintiles de revenu
    pauvre       = if_else(revenu_pc < 1.90 * 365, 1L, 0L)  # Seuil BM en USD
  ) |>
  group_by(region, milieu) |>
  summarise(
    n          = n(),
    revenu_moy = weighted.mean(revenu_pc, poids, na.rm = TRUE),
    tx_pauvrete = weighted.mean(pauvre, poids, na.rm = TRUE),
    .groups = "drop"
  ) |>
```

```
  arrange(desc(tx_pauvrete))
```

2. Gestion des valeurs manquantes

Les données d'enquêtes africaines présentent souvent des taux de valeurs manquantes élevés pour certaines variables sensibles (revenus, dépenses alimentaires détaillées, statut foncier). Il est essentiel de distinguer les types de missingness:MCAR (manquant complètement au hasard), MAR (manquant aléatoirement conditionnellement à d'autres variables) et MNAR (manquant non aléatoirement), car chaque type appelle une stratégie différente.

2.1 Exploration et visualisation des NA

```
library(naniar)

# Résumé des valeurs manquantes
miss_var_summary(menages) |> filter(pct_miss > 0)

# Matrice de co-occurrence des NA:quelles variables sont manquantes ensemble ?
gg_miss_upset(menages, nsets = 8)

# Pattern spatial des NA dans le dataset
vis_miss(menages, sort_miss = TRUE)
```

2.2 Imputation des valeurs manquantes

Pour les analyses exploratoires et descriptives, une imputation simple (médiane, mode) peut suffire. Pour la modélisation, une imputation multiple par équations chaînées (MICE) est recommandée car elle préserve l'incertitude liée aux valeurs manquantes.

```
library(mice)

# Imputation multiple (M = 5 jeux de données imputés)
imp <- mice(
  menages |> select(revenu, age_cm, educ_cm, taille_menage, milieu, region),
  m     = 5,          # Nombre de jeux imputés
  maxit = 20,         # Itérations de convergence
  method = "pmm",     # Predictive Mean Matching (recommandé pour variables
continues)
  seed  = 42
)

# Vérifier la convergence
plot(imp)

# Analyser sur les 5 jeux imputés et combiner (règles de Rubin)
```

```
fit_imputed <- with(imp, lm(log(revenu) ~ age_cm + educ_cm + taille_menage +
milieu))
pool(fit_imputed) |> summary()
```

3. Recodage et transformation des variables

Le recodage est l'une des tâches les plus fréquentes en nettoyage de données. Les enquêtes ménages utilisent souvent des codes numériques pour des variables qualitatives, des libellés incohérents, ou des échelles à inverser.

```
menages <- menages |>
  mutate(
    # Recodage avec case_when (plus flexible que if_else)
    milieu_label = case_when(
      milieu == 1 ~ "Urbain",
      milieu == 2 ~ "Rural",
      TRUE        ~ NA_character_
    ),

    # Création d'une variable catégorielle ordonnée
    niveau_vie = case_when(
      revenu_pc < 500  ~ "Très pauvre",
      revenu_pc < 1000 ~ "Pauvre",
      revenu_pc < 2000 ~ "Intermédiaire",
      TRUE             ~ "Non pauvre"
    ) |> factor(levels = c("Très pauvre", "Pauvre", "Intermédiaire", "Non pauvre"),
                ordered = TRUE),

    # Manipulation de chaînes avec stringr
    region_clean = region |>
      str_to_title() |>          # Première lettre en majuscule
      str_replace_all("_", " ") |>
      str_trim(),

    # Manipulation de dates avec lubridate
    annee_enquete = year(date_enquete),
    mois_enquete  = month(date_enquete, label = TRUE, abbr = FALSE)
  )
```

4. Restructuration et jointures

4.1 Passage format long ↔ format large

Les données d'enquêtes sont souvent distribuées en format large (une ligne par ménage, plusieurs colonnes pour les dépenses alimentaires par groupe). Certaines analyses nécessitent un format long (une ligne par ménage et par groupe alimentaire). tidyr permet de passer facilement d'un format à l'autre.

```
# Format large → format long
depenses_long <- depenses_large |>
  pivot_longer(
    cols      = starts_with("dep_"),    # Colonnes à transformer
    names_to  = "groupe_alim",          # Nouvelle colonne:nom du groupe
    values_to = "montant",              # Nouvelle colonne:valeur
    names_prefix = "dep_"               # Supprimer le préfixe
  )

# Format long → format large
depenses_large <- depenses_long |>
  pivot_wider(
    id_cols    = hhid,
    names_from = groupe_alim,
    values_from = montant,
    names_prefix = "dep_",
    values_fill  = 0                    # Remplacer les NA par 0
  )
```

4.2 Jointures entre tables

Les enquêtes ménages sont généralement distribuées en plusieurs tables (ménages, individus, dépenses, agriculture) liées par des identifiants. Les jointures permettent de combiner ces informations.

```
# Table ménages:une ligne par ménage
# Table individus:une ligne par individu (plusieurs par ménage)

# Joindre les caractéristiques du chef de ménage aux données ménage
chefs_menage <- individus |>
  filter(rang == 1) |>   # Rang 1 = chef de ménage
  select(hhid, sexe_cm = sexe, age_cm = age, educ_cm = niveau_educ)

menages_enrichis <- menages |>
  left_join(chefs_menage, by = "hhid") |>        # Conserver tous les ménages
  left_join(depenses_agregees, by = "hhid") |>
  anti_join(menages_exclus, by = "hhid")         # Exclure les ménages problématiques

# Vérification:le nombre de lignes ne doit pas avoir changé
nrow(menages_enrichis) == nrow(menages)          # Doit retourner TRUE
```

5. Bonnes pratiques de documentation du code

Un code bien documenté est aussi important que des résultats corrects. Dans un projet collaboratif ou pour la reproductibilité à long terme, les futurs utilisateurs de votre code (y

compris vous-même dans 6 mois) doivent pouvoir comprendre vos choix sans avoir à vous contacter.

- Commenter les choix non évidents:pourquoi ce filtre ? pourquoi ce seuil ?
- Utiliser des noms de variables explicites (tx_pauvrete plutôt que tp, revenu_pc plutôt que rpc)
- Structurer les scripts avec des sections numérotées et délimitées (# 1. IMPORT ====)
- Conserver les données brutes intactes — toujours travailler sur des copies
- Créer un dictionnaire des variables dans un fichier Excel ou Markdown dédié

A faire 3
1. Appliquer le pipeline dplyr complet au jeu de données EHCVM (filter, select, mutate, summarise)
2. Explorer et visualiser les valeurs manquantes avec naniar
3. Réaliser une imputation simple (médiane) pour les variables à faible taux de NA
4. Recoder les variables catégorielles en facteurs avec labels propres
5. Joindre les tables ménages, individus et dépenses du fil rouge
6. Livrable:Note de cadrage (2 pages) — problématique, données, plan d'analyse